

Operadores lógicos **and**, **or** e **not** em Python, são fundamentais para construir lógicas de decisão em seus programas. Eles operam sobre valores booleanos (True ou False) ou expressões que resultam em valores booleanos (como comparações: `x > 5`, `nome == "Maria"`).

Operador **and** (E Lógico)

Sintaxe: `expressao1 and expressao2`

Funcionamento: Retorna True somente se AMBAS `expressao1` E `expressao2` forem True.

Se qualquer uma delas (ou ambas) for False, o resultado é False.

expressao1	expressao2	expressao1 and expressao2
True	True	True
True	False	False
False	True	False
False	False	False

Exemplo: para entrar em um show, você precisa ter "idade permitida" E "ter ingresso". Se faltar um dos dois, você não entra.

Em Python:

```
idade = int(input("Digite sua idade:"))
tem_ingresso = True
pode_entrar_festa = (idade >= 18) and tem_ingresso
# (True) and True => True
print(f"Pode entrar na festa? {pode_entrar_festa}") # Saída: Pode entrar na festa? True

# =====
idade = 16
tem_ingresso = True
pode_entrar_festa = (idade >= 18) and tem_ingresso
# (False) and True => False
print(f"Pode entrar na festa? {pode_entrar_festa}") # Saída: Pode entrar na festa? False
```

Entendimento com and:

Se a `expressao1` for False, o Python nem avalia a `expressao2`, porque o resultado do **and** já será False de qualquer maneira. Isso pode ser útil para evitar erros ou processamento desnecessário.

```
x = 0
# Se x for 0, a segunda parte (10 / x) nunca será executada, evitando ZeroDivisionError
if x != 0 and (10 / x) > 2:
    print("Condição atendida")
else:
    print("Condição não atendida ou x é zero") # Saída: Condição não atendida ou x é zero
```

Operador or (OU Lógico)

Sintaxe: expressao1 or expressao2

Funcionamento: Retorna True se PELO MENOS UMA das expressões (expressao1 OU expressao2) for True. Só retorna False se AMBAS forem False.

Tabela:

expressao1	expressao2	expressao1 and expressao2
True	True	True
True	False	True
False	True	True
False	False	False

Exemplo: Para obter um desconto, você pode ser "estudante" OU "idoso". Se atender a pelo menos um desses critérios, você ganha o desconto.

Em Python:

```
eh_estudante = False
eh_idoso = True

tem_desconto = eh_estudante or eh_idoso
# False or True => True
print(f"Tem direito a desconto? {tem_desconto}") # Saída: Tem direito a desconto? True

# =====
eh_estudante = False
eh_idoso = False
tem_desconto = eh_estudante or eh_idoso
# False or False => False
print(f"Tem direito a desconto? {tem_desconto}") # Saída: Tem direito a desconto? False
```

Short-Circuit (Curto-Circuito) com or: Se a expressao1 for True, o Python nem avalia a expressao2, porque o resultado do or já será True de qualquer maneira.

```
tem_cupom = True

# Se tem_cupom for True, a função verifica_saldo() não é chamada
# def verifica_saldo():
#     print("Verificando saldo...")
#     return False # Exemplo
#
# if tem_cupom or verifica_saldo():
#     print("Pode comprar com desconto ou tem saldo.")
# else:
#     print("Sem cupom e sem saldo.")
```

Operador not (NÃO Lógico)

Sintaxe: not expressao

Funcionamento: É um operador unário (age sobre uma única expressão). Ele inverte o valor booleano da expressão.

Se a expressão for True, not expressao é False.

Se a expressão for False, not expressao é True.

Tabela:

expressao	not expressao
True	False
False	True

Exemplo: Se "está chovendo" é True, então "NÃO está chovendo" é False.

Em Python:

```
esta_chovendo = True
levar_guarda_chuva = esta_chovendo
print(f"Levar guarda-chuva? {levar_guarda_chuva}") # Saída: Levar guarda-chuva? True

nao_esta_chovendo = not esta_chovendo
# not True => False
print(f"Não está chovendo? {nao_esta_chovendo}") # Saída: Não está chovendo? False

porta_aberta = False
porta_fechada = not porta_aberta
# not False => True
print(f"A porta está fechada? {porta_fechada}") # Saída: A porta está fechada? True

# =====
```

Precedência dos Operadores Lógicos

Assim como na matemática, os operadores lógicos têm uma ordem de precedência (qual é avaliado primeiro):

not (é avaliado primeiro)

and (é avaliado depois do not)

or (é avaliado por último)

Os operadores de comparação (==, !=, >, <, >=, <=) são avaliados antes dos operadores lógicos.

Exemplo de Precedência e Uso de Parênteses:

```
a = True
b = False
c = True

resultado1 = a or b and c
# Primeiro b and c: False and True => False
# Depois a or False: True or False => True
print(f'Resultado 1 (a or b and c): {resultado1}') # Saída: True

resultado2 = (a or b) and c
# Primeiro (a or b): True or False => True
# Depois True and c: True and True => True
print(f'Resultado 2 ((a or b) and c): {resultado2}') # Saída: True

# Exemplo mais complexo
idade = 25
salario = 3000
tem_filhos = False

# Condição: ser maior de 18 E (ter salário acima de 2500 OU não ter filhos)
aprovado_emprestimo = idade > 18 and (salario > 2500 or not tem_filhos)
# idade > 18          => True
# salario > 2500       => True
# not tem_filhos       => not False => True
# (salario > 2500 or not tem_filhos) => (True or True) => True
# idade > 18 and (True)   => True and True => True
print(f'Aprovado para empréstimo? {aprovado_emprestimo}') # Saída: True
```

Neste último exemplo, os parênteses () são usados para garantir que a operação or seja realizada antes da operação and externa, alterando a precedência padrão. Usar parênteses, mesmo quando não estritamente necessário pela precedência, pode tornar seu código muito mais fácil de ler e entender.

Resumo:

A and B: Verdadeiro só se A e B forem verdadeiros.

A or B: Verdadeiro se A for verdadeiro, ou se B for verdadeiro, ou se ambos forem verdadeiros.

not A: Inverte o valor de A (True vira False, False vira True).

Esses operadores são os blocos de construção para criar fluxos de controle complexos em seus programas, permitindo que eles tomem decisões baseadas em múltiplas condições.